Computación

Clase 10

Temas

- Vectores paralelos
- Índice con significado
- Procesamiento elemental de datos

Vectores paralelos

Retomamos el ejemplo de la clase anterior en el que almacenábamos diferentes atributos (características) de un conjunto de datos.
La siguiente tabla presenta datos de un conjunto de N piezas metálicas numeradas de 1 a N, sometidas a diferentes tratamientos anticorrosión (A a E):

Nro	Tipo Metal	Peso	Tratamiento	Efectividad (en %)
1	A1	3,4	D	12
2	A2	2,7	A	19,2
3	B3	0,87	A	22
4	B7	1,9	D	0,76
5	A2	3,2	C	34

Vectores paralelos

Cada atributo tiene un tipo diferente, por lo tanto, definimos un arreglo para cada uno.

Nro (vec. de byte)	Tipo Metal (vec. de string[2])	Peso (vec. de real)	Trat (vec. de char)	Efectividad (vec. de real)

Tienen la misma cantidad de elementos, de diferente tipo.

Las componentes con **igual índice** conforman una **unidad de información**: una pieza.

El **índice** determina el número de elemento, por lo que el vector asociado al Nro no será necesario

Ejemplo

Se desea obtener:

- un listado de los Nros de Piezas y el Peso de las que tuvieron con el tratamiento D un porcentaje de efectividad mayor al 15%; y el peso promedio de las piezas que no se sometieron al tratamiento A

```
Program Ejemplo;
Type
   TVS=array[1..100] of string[2];
   TVR=array[1..100] of real;
   TVC=array[1..100] of char;

 Var {variables del programa principal}
   VTM:TVS; VEf, VPeso:TVR; VTrat:TVC;
   N:word;
   P:real;
```

```
Procedure LeeVectores(Var VTM:TVS; Var VPeso,
VEfect:TVR; Var VTrat:TVC; Var N:word);
Var
  i: word;
Begin
 Write('Cant. de piezas');  Readln(N);
 For i:=1 to N do
  Begin
     Write(‘Tipo Metal ');  Readln(VTM[i]);
     Write('Peso ');  Readln(VPeso[i]);
     Write(‘Efectividad ');  Readln(VEfect[i]);
     Write( 'Tratamiento '); Readln(VTrat[i]);
  End;
End;
```

```
Procedure Listado(Vpeso, VEfect:TVR; VTrat:TVC;
       N:word);
Var
  i: word;
Begin
 Write('Nros de Piezas y  Tratamiento');
 For i:=1 to N do
    If (VTrat[i]='D') and (VEfect[i]>15) then
      Writeln(i, VPeso[i]:6:2);
End;
```

```
Function Promedio(Vpeso: TVR; VTrat:TVC;  N:word):real;
Var
  I, Cont: word;
  S:real;
Begin
 S:=0;  Cont:=0;
 For i:=1 to N do
   If VTrat[i] <> 'A' then
       Begin
         Cont:= Cont + 1;
         S:=S + VPeso[i];
       End;
 If Cont <> 0 then
   Promedio:= S/Cont
 Else
   Promedio:=0
End;
```

```
Begin {programa principal}
 LeeVectores(VTM, VPeso, VEf, VTrat, N);
 Listado(VPeso, VEf, VTrat, N);
 P:= Promedio(VPeso, VTrat, N);
 If P = 0 then
  Writeln(‘todos las piezas se sometieron al
tratamiento A')
 Else
  Writeln('el promedio de peso de las piezas no
sometidas al tratamiento A es', P:8:2)
End.
```

Índice con significado

Como ya hemos visto, la estructura del vector permite el acceso directo a una componente determinada, indicando una posición que en general está asociada a alguna característica o atributo de la información tratada, es decir tiene un ***significado*** *dentro del ámbito del problema planteado.*

En el ejercicio anterior al NO almacenar el número de pieza y considerando que el mismo era un valor entre 1 y N, hemos asociado el índice a la información de las piezas (índice con significado)

Ejemplo

En un edificio de N pisos, se desea evaluar el uso del ascensor en relación a la cantidad de personas y al piso al cual se dirigen. Para ello durante un día se ha grabado en cada línea del archivo cada parada del ascensor:

- Piso
- Cantidad (de personas que bajan en dicho piso).

Se pide leer los datos del archivo para informar:

a. Para cada piso el promedio de personas que bajaron en él.

b. Cuál fue el piso con más paradas (no importa la cantidad de personas).

```
Program Ascensor;

Type
   TV=array[1..50] of word;
Var
Personas, Paradas:TV;
N: byte;

Procedure IniciaV(Var V:TV; N:byte);
Var
    i: Byte;
Begin
   For i:= 1 to N do
      V[i]:=0;
End;
```

```
Procedure Contabiliza(Var Personas, Paradas:TV;
N:byte);
Var
    Piso, Cant: Byte;
    Arch:text;
Begin
 IniciaV(Personas, N);  IniciaV(Paradas, N);
 Assign(Arch, 'Movimientos.txt'); Reset(Arch);
 While Not Eof(Arch)  do
     Begin
        Readln(Arch, Piso, Cant);
        Personas[Piso]:= Personas[Piso] + Cant;
        Paradas[Piso]:= Paradas[Piso] + 1;
     End;
 Close(Arch);
End;
```

```
Procedure Promedio( Personas, Paradas:TV; N:byte);
Var
    i: Byte;
Begin
 Writeln('Piso Promedio de personas por viaje');
 For i:= 1 to N do
    If Paradas[i]<> 0 then
      Writeln(i:3, Personas[i] div Paradas[i]:12)
    Else
      Writeln(i:3, 0:12)
End;
```

```
Function MaxPiso(V:TV; N:byte):byte;
Var
  i, Pos:byte;
begin
 Pos:=1;
 for i:=2 to N do
   if V[i]>V[Pos] then
     Pos:=i;
 MaxPiso:= Pos;
End;
```

```
Begin  {programa principal}
 Writeln('Ingrese cantidad de pisos');
 Readln(N);
 Contabiliza(Personas, Paradas, N);
 Promedio(Personas, Paradas, N);
 Writeln('El piso con mayor cantidad de viajes
es',MaxPiso(Paradas,N))
End.
```

Procesamiento elemental de datos : Búsqueda

- *Es el proceso de localización de un elemento de una lista o conjunto de elementos (arreglos).*
- *Resulta más eficiente si la lista está ordenada.*
- *Resulta conveniente ordenar el arreglo si se realizan búsquedas con mucha frecuencia.*

Procesamiento elemental de datos : Búsqueda

Siendo el arreglo una estructura de acceso secuencial y a su vez de acceso directo, permite implementar:

- Búsqueda secuencial (estructura desordenada/ordenada)
- Búsqueda binaria (solo para estrctura ordenada)

Búsqueda secuencial

- La búsqueda secuencial es la técnica más simple para buscar un elemento en un arreglo.

- Consiste en recorrer el arreglo elemento a elemento e ir comparando con el valor buscado (clave).

- Se comienza con el primer elemento del arreglo y se observa un elemento tras otro hasta que se encuentra el elemento buscado o se han visto todos los elementos. (**Estructura no ordenada**)

- Se comienza con el primer elemento del arreglo y se observa un elemento tras otro hasta que se encuentra el elemento buscado o uno mayor. (**Estructura ordenada de modo ascendente**)

Búsqueda secuencial de un determinado valor en un arreglo

Función entera que busca un valor X en un arreglo V de N elementos **desordenados**, devuelve la posición donde lo encuentra o N+1 si no está en el arreglo.

```
Function BuscaSecuencial ( V:TV; N:byte; X:integer ):byte;
Var
  i: byte;
Begin
 i:=1;
 While (i<= N) and (X <> V[i]) do
     i:= i+1;
 BuscaSecuencial := i;
End;
```

Búsqueda secuencial de un determinado valor en un arreglo

Función booleana que busca un valor X en un arreglo V de N elementos **desordenados**, devuelve true o false, según si el elemento está o no en el arreglo.

```
Function BuscaSecuencial(V:TV; N:byte; X:integer ):boolean;
Var
  i: byte;
Begin
 i:=1;
 While (i<= N) and (X <> V[i]) do
     i:= i+1;
 BuscaSecuencial := i<=N;
End;
```

Búsqueda secuencial de un determinado valor en un arreglo

Función booleana que busca un valor X en un arreglo V de N elementos **ordenados**, devuelve true o false, según si el elemento está o no en el arreglo.

```
Function BuscaSecuencial ( V:TV; N:byte; X:integer ):byte;
Var
  i: byte;
Begin
 i:=1;
 While (i<= N) and (X > V[i]) do
     i:= i+1;
 BuscaSecuencial := (i<=N) and (X = V[i]);
End;
```

Búsqueda binaria (sólo para vectores ordenados)

- El proceso comienza comparando el elemento central del arreglo con el valor buscado. Si ambos coinciden finaliza la búsqueda
- Si no ocurre así, el elemento buscado será mayor o menor en sentido estricto que el central del arreglo.
- Si el elemento buscado es mayor se procede a hacer búsqueda binaria en el subarreglo superior.
- Si el elemento buscado es menor que el contenido del elemento central, se debe cambiar el segmento a considerar al segmento que está a la izquierda de tal sitio central.

```
Function BusquedaBinaria ( V:TV; N:byte; x:integer ):boolean;
Var
    Medio, Pri, Ult: byte;
Begin
 Pri := 1;
 Ult := N;
 Medio:=(Pri + Ult) DIV 2;
 While (Pri <Ult) and (x <> V[ Medio ]) do
     Begin
       If  x < V[ Medio ] then
         Ult := Medio - 1
       Else
          Pri:= Medio + 1;
       Medio:=(Pri + Ult) DIV 2;
     End;
 BusquedaBinaria:=  x = V[ Medio]
End;
```

Procesamiento elemental de datos : ordenación

<u>Clasificación u ordenación</u> de datos es el proceso de reorganizarlos en una secuencia determinada.

- ✓ números en orden ascendente o descendente
- ✓ nombres en orden alfabético.

El propósito principal de un ordenamiento es el de facilitar las búsquedas (numérico, alfabético, ascendente o descendente).

Método de ordenamiento ascendente por intercambio

El método consiste en ir colocando en forma progresiva y ascendente los valores mínimos al inicio de la estructura.

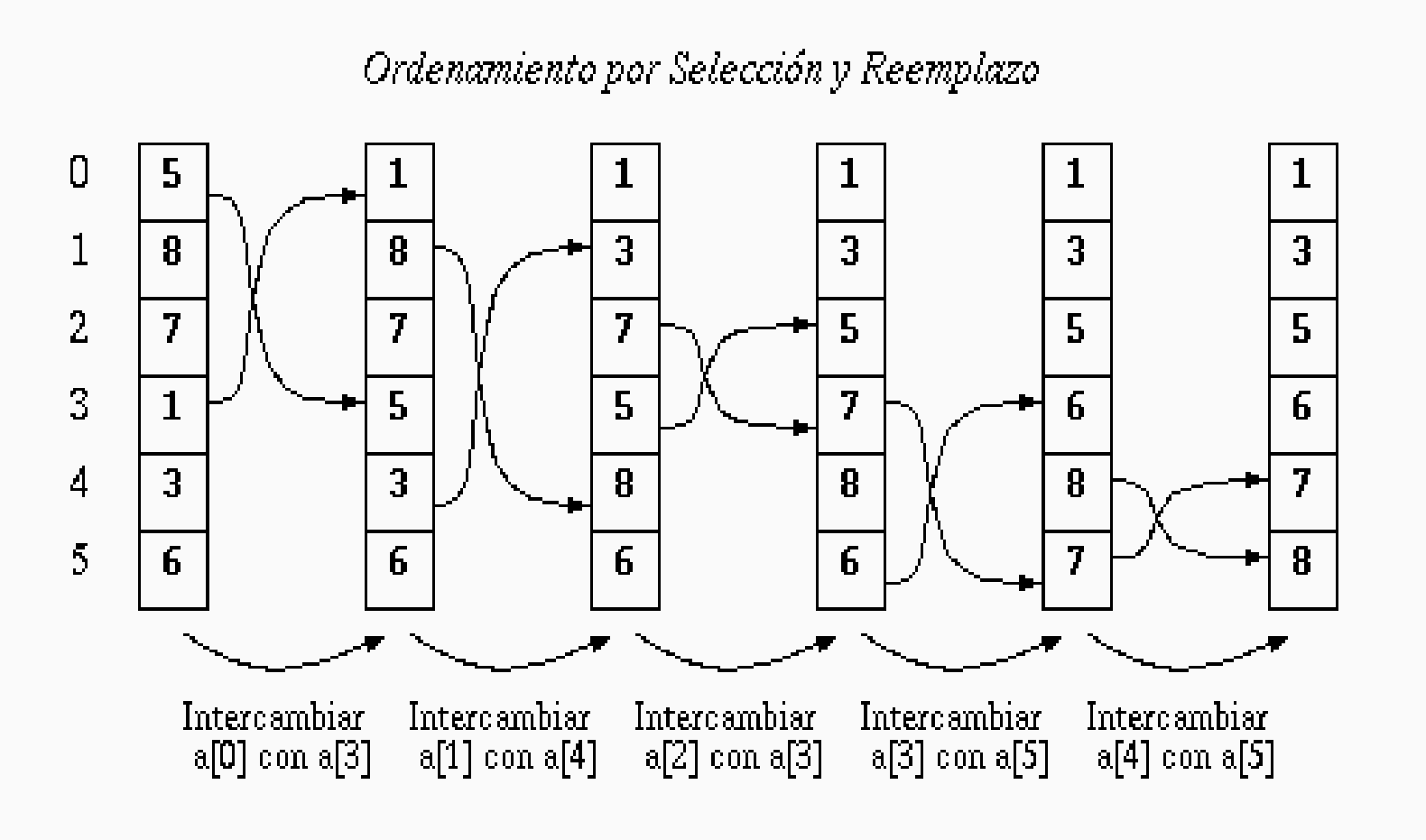
Veamos a continuación el modo en que se lleva a cabo.

Intercambio

2	4	5	-2	3	8	15	9
-2	4	5	**2**	3	8	9	15
-2	2	5	4	**3**	8	9	15
-2	2	3	4	5	8	9	15
-2	2	3	4	5	8	9	15
-2	2	3	4	5	8	9	15
-2	2	3	4	5	8	9	15

Notar que sigue recorriendo los subarreglos, y no detecta que ya esta ordenado

Ejemplo


Ordenamiento por Selección y Reemplazo
0
1
2
3
4
5
5 8 7 1 3 6
1 8 7 5 3 6
1 3 7 5 8 6
1 3 5 7 8 6
1 3 5 6 8 7
1 3 5 6 7 8
Intercambiar a[0] con a[3]
Intercambiar a[1] con a[4]
Intercambiar a[2] con a[3]
Intercambiar a[3] con a[5]
Intercambiar a[4] con a[5]

Es un método sencillo de implementar, pero ineficiente.

```
Type
  TV=array[1..100] of integer;
……————
Procedure Ordena(Var V:TV; N:byte);
Var
  i,k:byte;
  aux:integer;
Begin
For k:=1 to N-1 do {N-1 pasadas}
  For i:=k+1 to N do  {elementos de la pasada  k-esima}
      If  V[k]>V[i] then
         begin
            Aux:=V[k];
            V[k]:=V[i];
            V[i]:=aux
      End
End;
```

Ejercicio

A partir de un vector A de N elementos enteros ordenados, genera y devuelve un nuevo arreglo B donde se han eliminado los elementos repetidos (consecutivos iguales).

```
Program elimrepe ;
type
TV=array[1..100] of integer;
……
Procedure EliminaRepetidos(A:TV; N: byte; Var B:TV; Var M:Byte);
Var
  i:byte;
begin
M:=1;
B[1]:=A[1];
for i:=2 to N do
    if A[i]<> B[M] then
       begin
          M:=M+1;
          B[M]:=A[i];
       end;
end;
```

Notar que si el vector no estuviera ordenado, el algoritmo de eliminación de valores repetidos sería más complejo.